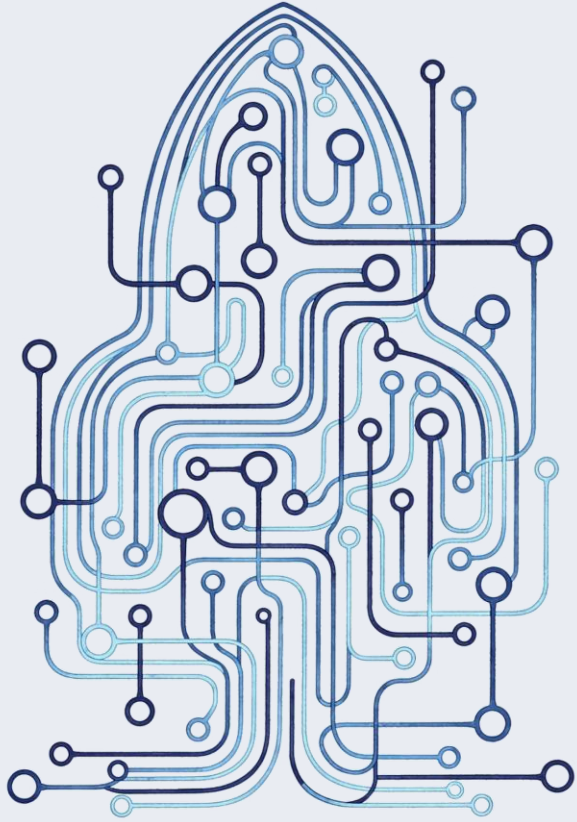




Análisis de la Eficiencia de Algoritmos y Paradigma Divide y Vencerás

Curso: Introducción a la Computación II · Carrera: Ingeniería en Sistemas

Objetivos de Aprendizaje

Al finalizar esta sesión, serás capaz de comprender, analizar y aplicar conceptos fundamentales sobre eficiencia algorítmica y el paradigma Divide y Vencerás.

1

Medir eficiencia

Comprender la importancia de medir la eficiencia de un algoritmo en términos de tiempo y espacio.

2

Paradigma D&V

Identificar las características del paradigma Divide y Vencerás y sus aplicaciones prácticas.

3

MergeSort

Analizar el funcionamiento de MergeSort y su implementación en C++.

4

Búsqueda Binaria

Aplicar la búsqueda binaria para resolver problemas de búsqueda eficiente sobre datos ordenados.

5

Notación Big O

Comparar complejidades temporales mediante notación Big O con gráficas comparativas.

¿Por Qué Importa la Eficiencia?

Búsqueda Desordenada

Revisar libro por libro en una pila caótica. En el peor caso, debes revisar **todos los elementos**. Tiempo: **$O(n)$**

Biblioteca Organizada

Ir directo a la sección correcta usando el índice. Divide el espacio de búsqueda repetidamente. Tiempo: **$O(\log n)$**

Factores clave

- **Tiempo de ejecución:** cuánto tarda en completarse
- **Uso de recursos:** memoria y procesador consumidos
- **Escalabilidad:** comportamiento al crecer los datos

❓ ¿Qué sucede cuando los datos aumentan de 100 a 1,000,000 elementos?



El Paradigma Divide y Vencerás

"Resolver un problema grande dividiéndolo en subproblemas más pequeños, resolver cada uno de forma independiente y combinar los resultados para obtener la solución final."



1. Dividir

El problema original se descompone en subproblemas más pequeños y manejables del mismo tipo.



2. Conquistar

Cada subproblema se resuelve de forma recursiva hasta llegar a un caso base trivial.



3. Combinar

Las soluciones parciales se ensamblan para construir la solución del problema original.

 **Ejemplo cotidiano:** Organizar una biblioteca dividiendo las tareas entre varios estudiantes — cada uno ordena su sección y luego se integran todos los resultados.

Aplicaciones del Paradigma Divide y Vencerás

Este paradigma es la base de algunos de los algoritmos más eficientes en Ciencias de la Computación. Sus aplicaciones abarcan desde ordenamiento hasta procesamiento de imágenes.



MergeSort

Algoritmo de ordenamiento $O(n \log n)$. Divide la lista hasta elementos individuales y fusiona en orden.



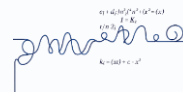
QuickSort

Selecciona un pivote y particiona el arreglo. Muy eficiente en la práctica con $O(n \log n)$ promedio.



Búsqueda Binaria

Divide el espacio de búsqueda a la mitad en cada paso. Requiere datos ordenados. Complejidad $O(\log n)$.



Multiplicación de Matrices

Algoritmo de Strassen divide matrices grandes en bloques más pequeños para multiplicación eficiente.

¿Qué es MergeSort?

Características principales

- **Algoritmo de ordenamiento** estable y confiable
- Basado en el paradigma **Divide y Vencerás**
- Divide recursivamente el arreglo hasta obtener **sublistas de un elemento**
- Fusiona (*merge*) las sublistas en orden creciente
- Complejidad garantizada: **$O(n \log n)$** en todos los casos

✓ Ideal para grandes volúmenes de datos y listas enlazadas.

Proceso visual: [38, 27, 43, 3]



Dividir

Subdividir

Fusionar

Resultado

Cada nivel del árbol de recursión divide el problema a la mitad, reduciendo la profundidad a $\log n$ niveles con n operaciones por nivel.

Código C++ de MergeSort

La implementación se divide en dos funciones: `merge()` que fusiona dos subarreglos ordenados, y `mergeSort()` que realiza la división recursiva.

```
#include <iostream>
using namespace std;

// Función que fusiona dos subarreglos ordenados
void merge(int arr[], int left, int mid, int right) {
    int n1 = mid - left + 1;    // Tamaño del subarreglo izquierdo
    int n2 = right - mid;       // Tamaño del subarreglo derecho

    int L[n1], R[n2];
    for(int i = 0; i < n1; i++) L[i] = arr[left + i];
    for(int j = 0; j < n2; j++) R[j] = arr[mid + 1 + j];

    int i = 0, j = 0, k = left;
    while(i < n1 && j < n2) {    // Comparar y fusionar
        if(L[i] <= R[j]) { arr[k] = L[i]; i++; }
        else { arr[k] = R[j]; j++; }
        k++;
    }
    while(i < n1) { arr[k] = L[i]; i++; k++; } // Copiar restantes
    while(j < n2) { arr[k] = R[j]; j++; k++; }
}

// Función recursiva principal
void mergeSort(int arr[], int left, int right) {
    if(left < right) {
        int mid = left + (right - left) / 2; // Calcular punto medio
        mergeSort(arr, left, mid);          // Dividir izquierda
        mergeSort(arr, mid + 1, right);      // Dividir derecha
        merge(arr, left, mid, right);        // Combinar resultados
    }
}
```



Dividir: `mid = left + (right-left)/2` calcula el punto medio sin desbordamiento.



Comparar: El bucle `while` compara elemento a elemento ambos subarreglos.



Copiar: Los elementos sobrantes se copian directamente al arreglo final.

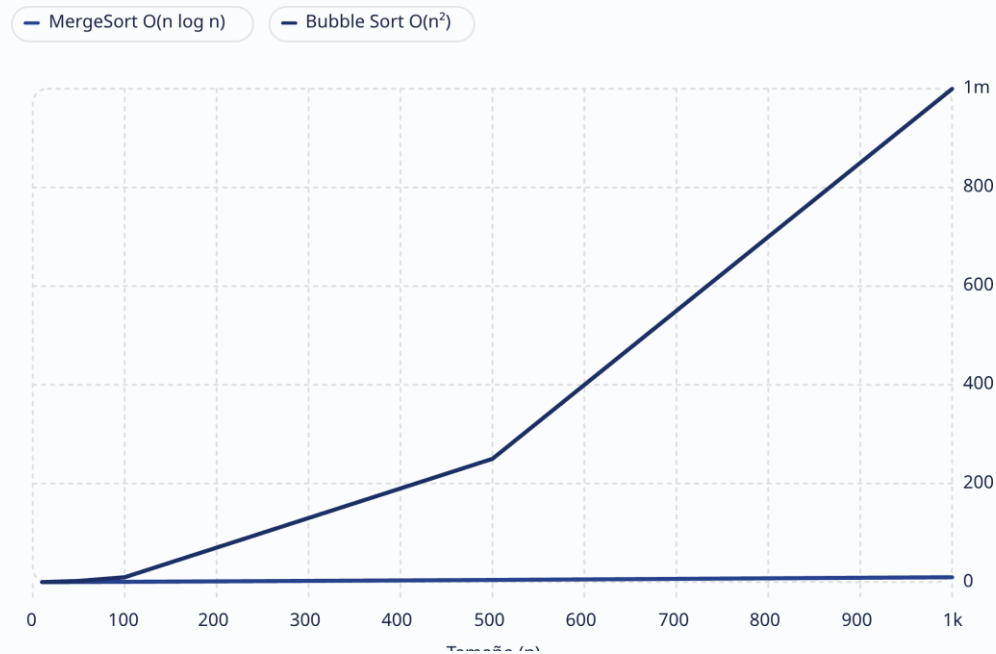
Complejidad de MergeSort

Tabla comparativa de casos

Caso	Complejidad	Detalle
Mejor caso	$O(n \log n)$	Lista ya ordenada
Caso promedio	$O(n \log n)$	Datos aleatorios
Peor caso	$O(n \log n)$	Lista invertida
Espacio	$O(n)$	Arreglos auxiliares

✓ **Ventaja clave:** MergeSort siempre garantiza $O(n \log n)$ sin importar el orden inicial de los datos.

MergeSort vs. Bubble Sort – Crecimiento de operaciones



Con $n=1,000$ elementos, Bubble Sort requiere ~1,000,000 operaciones mientras MergeSort solo ~10,000. La diferencia se vuelve dramática a mayor escala.

¿Qué es la Búsqueda Binaria?

Concepto fundamental

La búsqueda binaria es un método de búsqueda altamente eficiente que opera sobre **arreglos ordenados**. En cada iteración, **reduce el espacio de búsqueda exactamente a la mitad**, descartando la mitad que no puede contener el elemento buscado.

Requisito

Los datos deben estar **ordenados** previamente

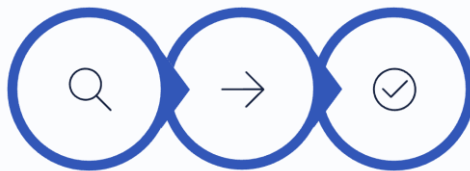
Estrategia

Comparar con el elemento del **punto medio**

Resultado

Reduce el problema a la **mitad** en cada paso

Búsqueda de 23 en: [2, 5, 8, 12, 16, 23, 38, 56, 72]



Comprobar 16

Comprobar 38

Encontrar 23

En solo 3 pasos se encuentra el elemento en una lista de 9 elementos. Una búsqueda lineal requeriría hasta 9 comparaciones en el peor caso.



Código C++ de Búsqueda Binaria

La implementación iterativa es simple y eficiente. Los punteros `izquierda` y `derecha` delimitan la zona activa de búsqueda, que se reduce a la mitad en cada iteración.

```
#include <iostream>
using namespace std;

int busquedaBinaria(int arr[], int n, int objetivo) {
    int izquierda = 0; // Límite inferior inicial
    int derecha = n - 1; // Límite superior inicial

    while(izquierda <= derecha) {
        // Calcular punto medio (evita desbordamiento)
        int medio = izquierda + (derecha - izquierda) / 2;

        if(arr[medio] == objetivo) //
```

Complejidad de la Búsqueda Binaria

Comparativa de búsquedas

Algoritmo	Complejidad	Ordenamiento requerido
Búsqueda Lineal	$O(n)$	No
Búsqueda Binaria	$O(\log n)$	Sí

❗ Con **1,000,000 elementos**, la búsqueda lineal puede tomar 1,000,000 comparaciones. La búsqueda binaria toma solo **20 comparaciones**.

20

Comparaciones máximas

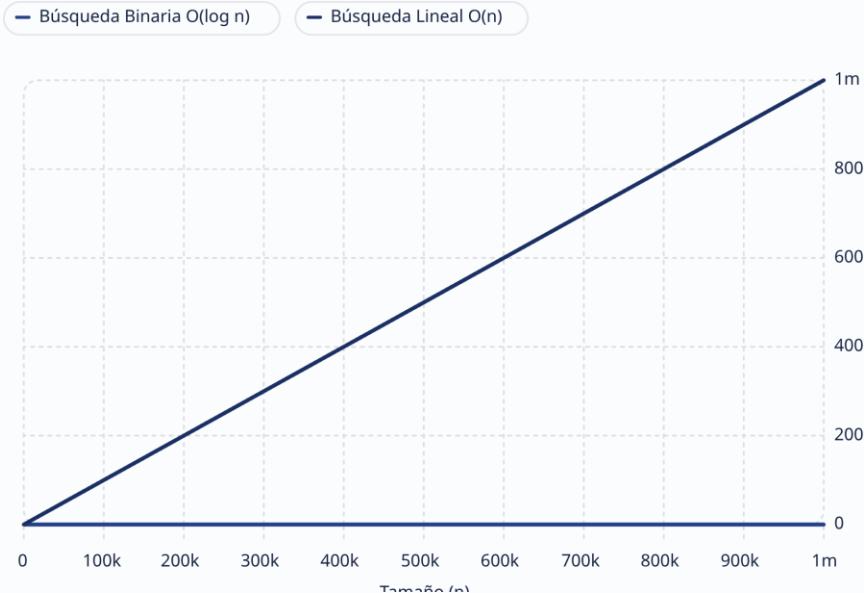
para $n = 1,000,000$

50x

Veces más rápida

que búsqueda lineal a gran escala

Crecimiento de operaciones: Lineal vs. Binaria



Comparación General de Algoritmos

Una visión integrada de todos los algoritmos estudiados permite tomar decisiones informadas sobre cuál utilizar según el problema, el tamaño de los datos y los recursos disponibles.

Algoritmo	Tipo	Mejor caso	Caso promedio	Peor caso	Espacio	¿Requiere orden?
Bubble Sort	Ordenamiento	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	No
MergeSort	Ordenamiento	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	No
Búsqueda Lineal	Búsqueda	$O(1)$	$O(n)$	$O(n)$	$O(1)$	No
Búsqueda Binaria	Búsqueda	$O(1)$	$O(\log n)$	$O(\log n)$	$O(1)$	Sí



✔ **MergeSort** es preferible a Bubble Sort para n grande, a pesar de usar más memoria auxiliar.



Búsqueda Binaria es dramáticamente más eficiente, pero requiere que los datos estén ordenados previamente.

Actividad Interactiva

Aplica los conceptos aprendidos resolviendo los siguientes ejercicios. Trabaja individualmente o en parejas y prepárate para socializar tus respuestas.

1

Ordenar con MergeSort

Ordena la siguiente lista aplicando la lógica de MergeSort paso a paso. Dibuja el árbol de recursión:

[45, 12, 78, 3, 56, 29, 91, 17]

Muestra cada nivel de división y cada paso de fusión.

2

Búsqueda Binaria

Dado el arreglo ordenado **[3, 7, 11, 18, 25, 32, 44, 58, 67, 79]**, encuentra el elemento **32**.

Registra en cada paso: índice medio, valor comparado y decisión tomada.

3

Identificar Complejidad

Para cada algoritmo a continuación, indica su complejidad temporal Big O:

- Recorrer todos los elementos de un arreglo
- Encontrar el máximo en un arreglo no ordenado
- Ordenar con MergeSort y luego buscar con búsqueda binaria



Pregunta de reflexión: ¿Cuándo NO conviene usar búsqueda binaria aunque los datos estén ordenados?

Conclusiones

Los conceptos aprendidos en esta sesión son fundamentales para el diseño de software eficiente y escalable. La elección del algoritmo correcto puede marcar la diferencia entre un sistema funcional y uno que colapsa ante grandes volúmenes de datos.



Eficiencia es esencial

En sistemas con millones de registros, la diferencia entre $O(n^2)$ y $O(n \log n)$ puede significar horas vs. segundos.



Divide y Vencerás

Permite resolver problemas complejos de forma elegante, reduciendo cada nivel del problema a tamaños manejables.



MergeSort: confiable

Garantiza $O(n \log n)$ en todos los casos. Es estable y predecible, ideal para datos críticos.



Búsqueda Binaria: poder logarítmico

Transforma búsquedas de millones de elementos en apenas 20 comparaciones. Un cambio de paradigma en eficiencia.



Pregunta final de reflexión: ¿Qué otros problemas cotidianos o de Ingeniería en Sistemas podrían resolverse aplicando el paradigma Divide y Vencerás?

Comparte tu respuesta.